**Informe técnico: decisiones de diseño de la base de datos de la red social estudiantil pascualina**

**1. Objetivo**

Este informe explica por qué la base de datos se diseñó como está y por qué se implementó en PostgreSQL. Parte del modelo Entidad-Relación (notación de Chen) con **7 entidades y 12 relaciones** y llega a un modelo físico de **13 tablas**, 18 claves foráneas y 12 índices adicionales. Los campos y tipos de cada tabla están en el documento Diccionarios de datos.

**2. Por qué PostgreSQL**

Se eligió **PostgreSQL**, administrado con **pgAdmin 4**, por estas razones:

- **Integridad referencial sólida.** Soporta claves foráneas con acciones CASCADE, RESTRICT y SET NULL, además de restricciones CHECK, UNIQUE y NOT NULL. Con ellas la base de datos hace cumplir por sí misma las cardinalidades y participaciones del modelo.
- **Concurrencia.** Su control MVCC permite que muchas personas lean y escriban a la vez sin bloquearse, algo típico de una red social.
- **Tipos adecuados al dominio.** TIMESTAMPTZ (fecha y hora con zona horaria), TEXT (texto de longitud libre) y BOOLEAN encajan directamente con publicaciones, eventos y mentorías.
- **Lógica dentro de la base de datos.** Permite funciones y triggers en PL/pgSQL; aquí se usan para numerar los comentarios de cada publicación.
- **Costo y herramientas.** Es de código abierto y sin licencias, con buena documentación. pgAdmin 4 permite ejecutar el script y generar el diagrama ER inverso (Generate ERD) para comprobar que el modelo físico coincide con el conceptual.

**Frente a MySQL:** PostgreSQL sigue de forma más estricta el estándar SQL y aplica las restricciones CHECK de manera consistente.

**Frente a una base NoSQL:** el dominio es muy relacional (seguimientos, membresías, asistencias, reacciones) y necesita integridad transaccional; un modelo de documentos obligaría a duplicar datos y a controlar a mano la coherencia.

**3. De modelo ER a modelo relacional**

| Elemento del modelo ER | Cómo quedó en la base de datos | Por qué |
|---|---|---|
| Entidad fuerte (estudiante, habilidad, interes, publicacion, grupo, evento) | Una tabla con clave primaria id_* | Es la forma estándar de representar una entidad con identidad propia |
| Entidad débil (comentario) | Tabla con PK compuesta (id_publicacion, id_comentario) | Un comentario no tiene sentido sin su publicación |
| Relación 1:N (publica, crea, organiza, asociado_a, escribe) | Clave foránea en la tabla del lado N | Evita crear tablas innecesarias |
| Relación M:N (tiene_habilidad, tiene_interes, reacciona, participa_en, asiste) | Tabla asociativa con PK compuesta por las dos FK | El modelo relacional no admite M:N directas |
| Relación recursiva M:N (sigue) | Tabla asociativa con dos FK a estudiante (id_seguidor, id_seguido) | Un estudiante cumple dos papeles en la misma relación |
| Relación identificadora (pertenece_a) | La FK es parte de la PK de comentario | Es lo que define a una entidad débil |
| Atributos de relación | Columnas de la tabla asociativa (nivel, rol, estado_asistencia, es_mentoria…) | Son datos de la relación, no de una sola entidad |

**Resultado:** 13 tablas = 7 de entidades + 6 asociativas (sigue, tiene_habilidad, tiene_interes, reacciona, participa_en, asiste).

**4. Decisiones sobre tipos de datos**

- **Identificadores:** INTEGER GENERATED ALWAYS AS IDENTITY. Es la forma estándar de SQL (mejor que SERIAL) y evita que alguien inserte un id a mano por error. Un INTEGER alcanza más de 2.000 millones de valores, más que suficiente para una comunidad universitaria.
- **Textos cortos:** VARCHAR(n) con longitudes ajustadas al dato (nombre y apellido 60, correo 120, nombre de evento 120, biografía y descripciones 500). Así se evita basura y se documenta el tamaño esperado.
- **Publicaciones y comentarios:** TEXT, porque su longitud es impredecible.
- **Fechas:** DATE cuando la hora no importa (nacimiento, ingreso, creación) y TIMESTAMPTZ cuando sí (publicaciones, reacciones, eventos). Guardar la zona horaria evita ambigüedades si los usuarios se conectan desde lugares distintos.

- **Semestre:** SMALLINT, porque solo toma valores de 1 a 12. **Es mentoría:** BOOLEAN, porque es sí o no.
- **Dominios cerrados** (tipo de publicación, categoría de grupo, tipo de evento, rol, nivel, estado de asistencia, tipo de reacción): VARCHAR con CHECK ... IN (...). Es más fácil de modificar que un ENUM cuando el catálogo cambie (por ejemplo, agregar un nuevo tipo de reacción).

**5. Claves primarias**

- **Entidades fuertes: clave sustituta autogenerada.** El correo institucional es una clave candidata y se protege con UNIQUE, pero no se usa como PK porque puede cambiar y es largo para repetirlo como FK en muchas tablas.
- **Tablas asociativas: PK compuesta.** La combinación de las dos FK impide duplicados (nadie se inscribe dos veces al mismo evento ni reacciona dos veces a la misma publicación) y no hace falta un id artificial.
- **Comentario: PK compuesta (id_publicacion, id_comentario).** id_comentario es la clave parcial: un consecutivo (1, 2, 3…) dentro de cada publicación. Un trigger BEFORE INSERT lo calcula (máximo actual + 1) y bloquea la fila de la publicación mientras tanto, para que dos comentarios simultáneos no reciban el mismo número.

**6. Claves foráneas y reglas de borrado**

La acción al borrar se eligió según el significado de cada relación:

| Regla | Dónde se aplica | Justificación |
|---|---|---|
| **ON DELETE CASCADE** | Comentarios de una publicación; publicaciones, comentarios, reacciones, habilidades, intereses, seguimientos, membresías y asistencias de un estudiante; membresías de un grupo; asistencias a un evento | Son datos que no tienen sentido sin su "padre"; dejarlos sería dejar registros huérfanos |
| **ON DELETE RESTRICT** | grupo.id_creador y evento.id_organizador | No se debe poder borrar a un estudiante que sostiene un grupo o un evento sin antes transferir esa responsabilidad |
| **ON DELETE SET NULL** | evento.id_grupo | Si un grupo desaparece, sus eventos se conservan como eventos independientes |
| **ON UPDATE CASCADE** | Todas las FK | Si alguna vez cambia una clave, el cambio se propaga solo |

Son **18 claves foráneas** en total.

**7. Participación y cardinalidad**

- **Participación total (1,1)** - columna FK **NOT NULL**: autor de una publicación, autor de un comentario, creador de un grupo y organizador de un evento. Es imposible que existan sin ese estudiante.
- **Participación parcial** de asociado_a - evento.id_grupo acepta **NULL**: un evento puede o no pertenecer a un grupo.
- **Cardinalidad M:N** - tablas asociativas.

**8. Otras reglas en la base de datos**

Estas reglas se pusieron en la base y no solo en la aplicación, para que nadie pueda saltarse la validación:

- **UNIQUE:** correo institucional; nombre de habilidad; nombre de interés.
- **CHECK:** semestre entre 1 y 12; fecha de nacimiento anterior a hoy; formato de correo válido; contenido de publicaciones y comentarios no vacío; fecha de fin de evento posterior a la de inicio; un estudiante no puede seguirse a sí mismo; listas cerradas de valores (tipo, categoría, rol, nivel, estado).
- **DEFAULT:** fecha y hora actuales en publicaciones, comentarios, reacciones e inscripciones; nivel basico; rol miembro; estado inscrito; mentoría FALSE.

**9. Índices**

PostgreSQL indexa automáticamente las PK y las columnas UNIQUE, pero no las claves foráneas. Sin índice, cada JOIN y cada borrado en cascada tendría que recorrer la tabla completa. Por eso se crearon 12 índices sobre las FK que no son el primer campo de una PK (por ejemplo reacciona.id_publicacion o asiste.id_evento). Además:

- publicacion (id_estudiante, fecha_hora DESC) - acelera la línea de tiempo de un autor, con lo más reciente primero.
- evento (fecha_hora_inicio) - acelera la consulta de próximos eventos.

**10. Validación**

El script incluye datos de prueba mínimos que recorren las relaciones principales: dos estudiantes, una publicación, dos comentarios (con numeración automática 1 y 2), un seguimiento como mentoría, un grupo con dos miembros, un evento asociado al grupo y una asistencia. Si alguna restricción estuviera mal definida, estas inserciones fallarían.

## 11. Conclusión

El modelo físico conserva fielmente el modelo conceptual: cada una de las 7 entidades y las 12 relaciones tiene una representación explícita. Las claves primarias y foráneas garantizan la coherencia de los datos; las restricciones CHECK, UNIQUE, NOT NULL y DEFAULT trasladan las reglas de negocio a la propia base; y los índices y el trigger de comentarios resuelven rendimiento y concurrencia. PostgreSQL es una elección adecuada por su integridad referencial, su manejo de concurrencia y su carácter abierto.